

EGCO 425 – Chapter 8

Regression

Linear & Logistic Regression

Classification and Regression Tree (CART)

Support Vector Regression (SVR)

References

- [1] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 5)
- [2] RapidMiner Doc, <https://docs.rapidminer.com/latest/studio/operators/>

Chapter Outline

⊕ Numeric prediction

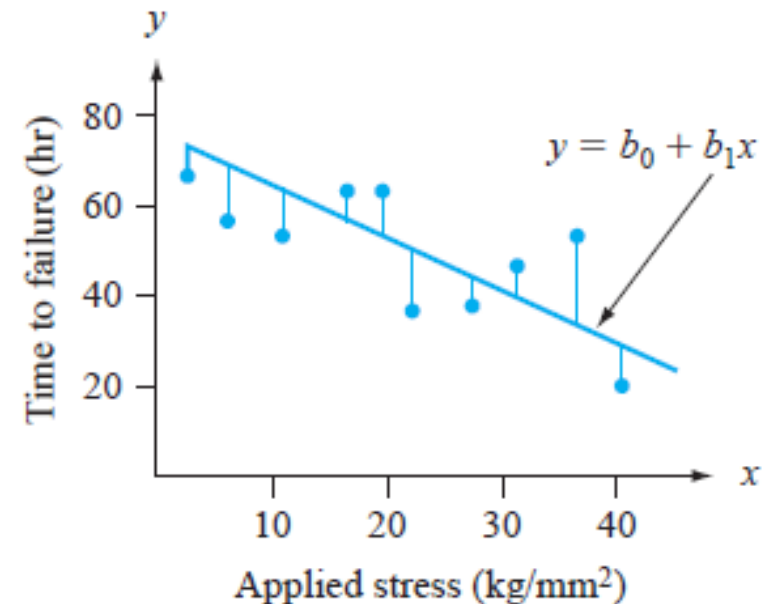
- ▶ Linear regression
- ▶ Extensions of classification methods
 - Classification and regression tree (CART)
 - Support vector regression (SVR)
 - Neural network
- ▶ Evaluation metrics

⊕ Classification

- ▶ Logistic regression

Function Fitting

- ⊕ Predict value or class label Y by using function $Y = f(X)$
 - ▶ X = set of attributes or predictors
 - ▶ Find $f(X)$ that fits training data
- ⊕ Function fitting approaches
 - ▶ Linear regression for numeric prediction (Y = numeric value)
 - ▶ Support vector regression for numeric prediction
 - ▶ Logistic regression for classification (Y = class label)



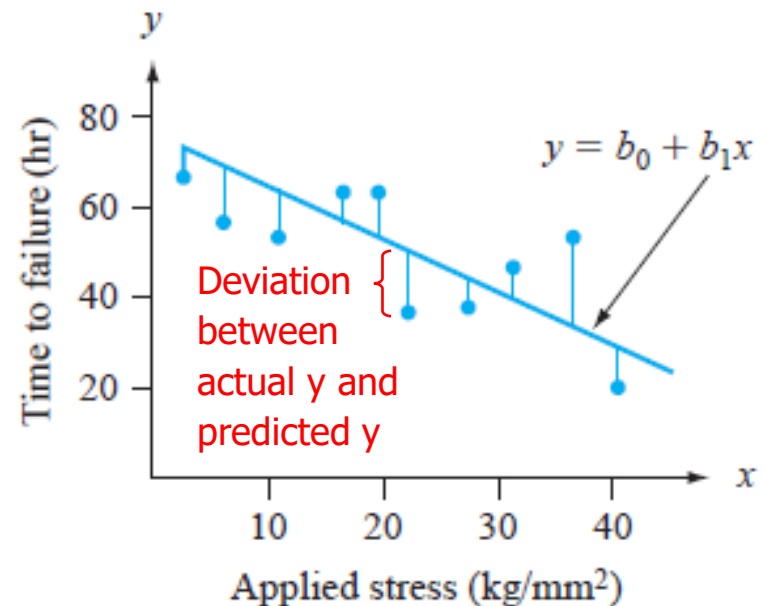
Simple Linear Regression

- ⊕ Consider only 1 predictor X and numeric target Y
 - ▶ Training data = $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- ⊕ Find a linear function $y = b_0 + b_1x$ that best fits the data
 - ▶ Among many possible lines, choose the one with the smallest sum of squared (vertical) deviation $\rightarrow$ lowest training error

$$b_1 \text{ (i. e. slope)} = \frac{\sum x_i y_i - \frac{(\sum x_i)(\sum y_i)}{n}}{\sum x_i^2 - \frac{(\sum x_i)^2}{n}}$$

$$= \text{correlation}(X, Y) \left(\frac{SD \text{ of } Y}{SD \text{ of } X} \right)$$

$$b_0 \text{ (i. e. intercept)} = \bar{y} - b_1 \bar{x}$$



Multiple Linear Regression

⊕ Consider multiple predictors or attributes $A_1, A_2, \dots, A_m$

⊕ Regression model is $y = b_0 + b_1A_1 + b_2A_2 + \dots + b_mA_m$

Using A_i to avoid confusion with x_i in previous slide (since x_i = individual data point)

⊕ Similar calculation to obtain regression coefficients $b_0, b_1, \dots, b_m$

⊕ Feature selection can be applied during the model fitting

- ▶ Most methods are based on statistical test on regression coefficients
- ▶ Collinear attributes (i.e. attributes that are correlated with each other) can be removed. As a result, only correlation between each individual attribute and Y is considered



Row No.	label	prediction(la...	a1	a2	a3	a4	a5	residual
1	88.016	121.137	0.637	8.442	2.116	2.772	8.448	-33.120
2	138.755	137.225	4.595	4.388	4.926	2.682	8.618	1.529
3	97.405	131.657	2.691	3.572	7.806	6.843	3.054	-34.252
4	140.767	255.464	6.226	8.381	0.351	3.639	6.234	-114.698
5	285.871	322.562	3.163	8.397	7.110	3.342	6.405	-36.690



If the model is good, residual or error should be normally distributed (indicating that errors are random), with mean being close to 0

In practice, the model can still work well even when this is violated

Attribute	Coefficient	Std. Error	Std. Coeff...	Tolerance	t-Stat	p-Value	Code
a1	31.736	2.114	0.463	1.000	15.009	0	****
a2	42.948	1.928	0.689	0.998	22.276	0	****
a3	23.773	2.000	0.369	1.000	11.888	0	****
a4	3.706	2.060	0.056	1.000	1.799	0.074	*
a5	-4.184	1.923	-0.068	0.999	-2.176	0.031	**
(Intercept)	-304.228	23.603	?	?	-12.889	0	****

LinearRegression

```

31.736 * a1
+ 42.948 * a2
+ 23.773 * a3
+ 3.706 * a4
- 4.184 * a5
- 304.228

```

Coefficient of each attribute (e.g. 31.736 for a1) = effect of that attribute on the target = attribute weight

- When a1 increases by 1 → target increases by 31.736
- All other attributes are held constant

T-Stat and p-Value = statistical test on coefficient → to confirm the significance of each attribute

- P-value close to 0 indicates high significance
- Or check #stars in the last column (more stars = higher significance)

Attribute	Coefficient	Std. Error	Std. Coeffic...	Tolerance	t-Stat	p-Value	Code
a1	31.736	2.114	0.463	1.000	15.009	0	****
a2	42.948	1.928	0.689	0.998	22.276	0	****
a3	23.773	2.000	0.369	1.000	11.888	0	****
a4	3.706	2.060	0.056	1.000	1.799	0.074	*
a5	-4.184	1.923	-0.068	0.999	-2.176	0.031	**
(Intercept)	-304.228	23.603	?	?	-12.889	0	****

LinearRegression

```

31.736 * a1
+ 42.948 * a2
+ 23.773 * a3
+ 3.706 * a4
- 4.184 * a5
- 304.228

```

Tolerance indicates collinearity between each attribute and the others

- Should be close to 1
- Tolerance < 0.2 → that attribute has severe collinearity with some others

PerformanceVector

```

PerformanceVector:
root_mean_squared_error: 85.060 +/- 0.000
root_relative_squared_error: 0.476
correlation: 0.882
squared_correlation: 0.778

```

Evaluating Regression Performance

⊕ Prediction error $e_i = y_i - \hat{y}_i$

▶ Root of mean squared error (RMSE)

- Average error
- Most common metric

$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{n}}$$

▶ Root of relative squared error (RRSE)

- Error w.r.t. variance of this data set
- Useful for comparing different models

$$RRSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}}$$

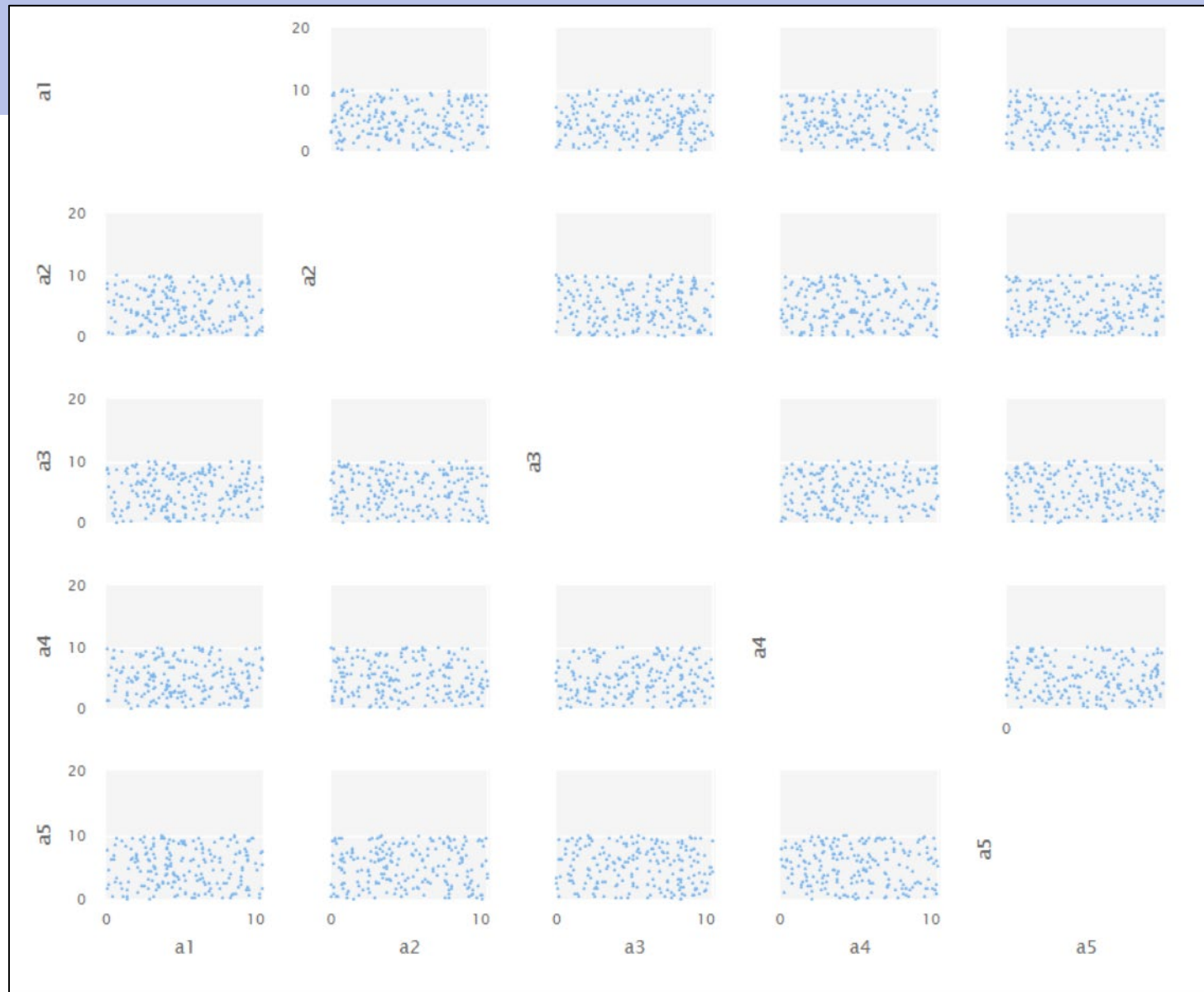
$$0 \leq RRSE \leq 1$$

⊕ Correlation

- ▶ Multiple correlation between all attributes and Y ($0 \leq r \leq 1$)
- ▶ Multiple coefficient of determination ($0 \leq r^2 \leq 1$)
 - Tell how much variation in Y can be explained by the model
 - Or how well the regression model fits the data

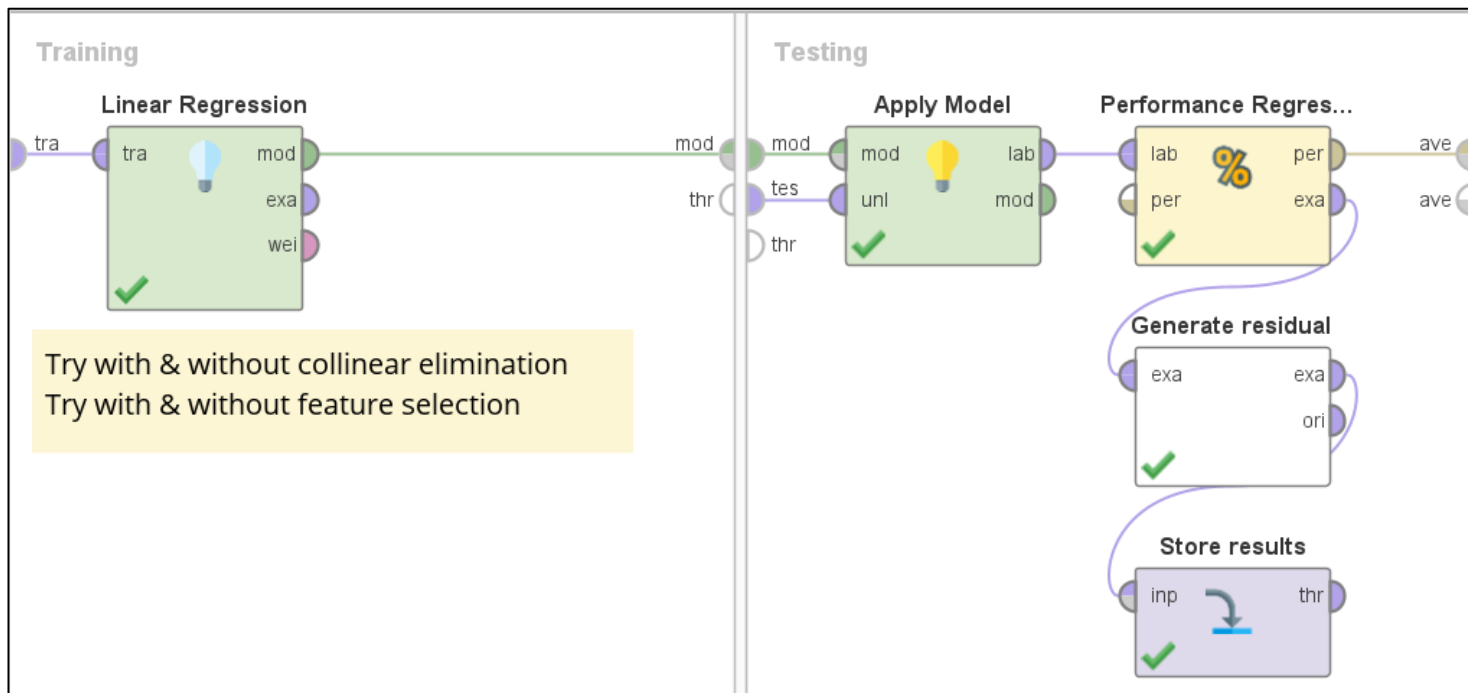
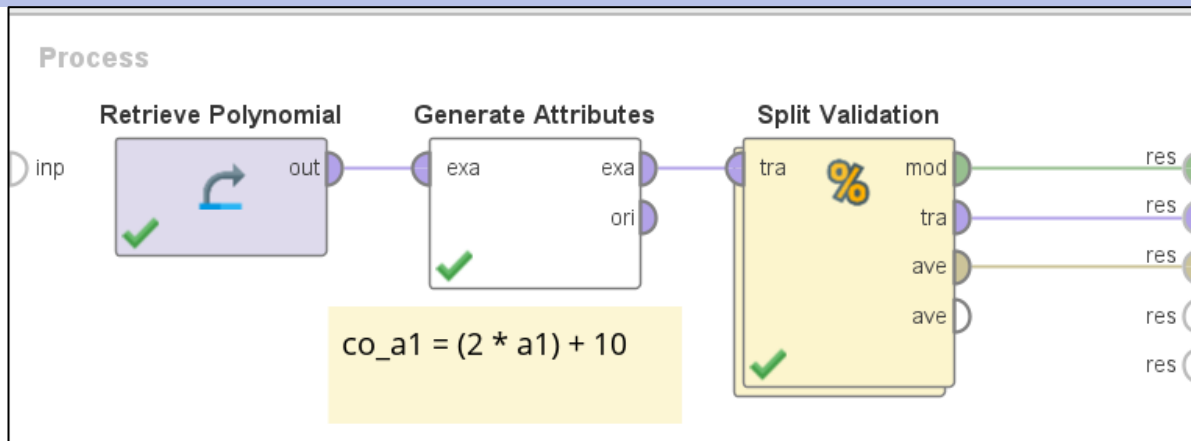
Collinearity

- ⊕ Any 2 attributes in the model are highly correlated or having linear relationship with each other
 - ▶ Not good for explanation. When 2 attributes change together, we may get wrong conclusion about the effect of each attribute on the target
 - ▶ Cannot hold one attribute constant in order to control the effect of the other → result in unreliable model
- ⊕ Multicollinearity means there are >2 highly correlated attributes
- ⊕ Preliminary check by scatter matrix



No collinearity among a_1 - a_5

Example (2) : linear regression with collinearity



No feature selection & no collinear feature elimination

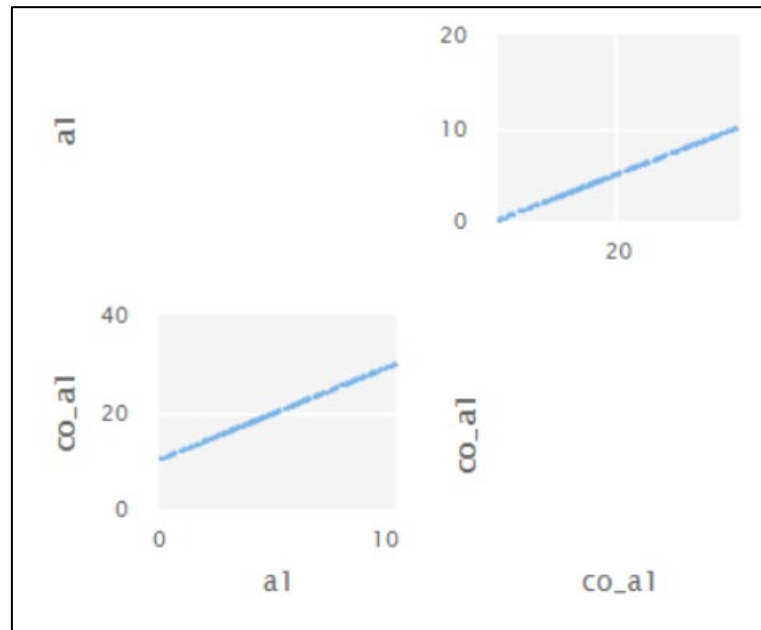
Attribute	Coefficient	Std. Error	Std. Coeffic...	Tolerance	t-Stat	p-Value	Code
a1	6.352	2.486	0.093	0.725	2.555	0.011	**
a2	42.948	1.929	0.689	0.998	22.260	0	****
a3	23.773	1.989	0.369	1.000	11.950	0	****
a4	3.706	2.050	0.056	1.000	1.807	0.072	*
a5	-4.184	1.916	-0.068	0.999	-2.184	0.030	**
co_a1	12.692	1.243	0.371	0.725	10.211	0	****
(Intercept)	-431.147	∞	?	?	-0	1	

PerformanceVector

```
PerformanceVector:
root_mean_squared_error: 85.060 +/- 0.000
root_relative_squared_error: 0.476
correlation: 0.882
squared_correlation: 0.778
```

Both a1 and co_a1 are significant attributes. But because they are highly correlated, their individual effect on the target is misleading

Seeing only tolerances (0.725), we may not know that they have collinearity



$a1$ and co_a1 are highly correlated
(collinearity exists between $a1$ and co_a1)

No feature selection & min collinear threshold = 0.75

Attribute	Coefficient	Std. Error	Std. Coeffic...	Tolerance	t-Stat	p-Value	Code
a2	42.948	1.928	0.689	0.998	22.276	0	****
a3	23.773	2.000	0.369	1.000	11.888	0	****
a4	3.706	2.060	0.056	1.000	1.799	0.074	*
a5	-4.184	1.923	-0.068	0.999	-2.176	0.031	**
co_a1	15.868	1.057	0.463	1.000	15.009	0	****
(Intercept)	-462.906	29.764	?	?	-15.553	0	****

PerformanceVector

```
PerformanceVector:
root_mean_squared_error: 85.060 +/- 0.000
root_relative_squared_error: 0.476
correlation: 0.882
squared_correlation: 0.778
```

*Performance is the same as before
because we can use just 1 attribute
instead of both*

Without the influence from a1, the effect of co_a1 on the target is clearer

Usually we don't set min collinear threshold this high because some attributes that don't have collinearity may also be eliminated

Greedy feature selection (try with & without collinear feature elimination)

Attribute	Coefficient	Std. Error	Std. Coeffic...	Tolerance	t-Stat	p-Value	Code
a2	42.948	1.928	0.689	0.998	22.276	0	****
a3	23.773	2.000	0.369	1.000	11.888	0	****
a4	3.706	2.060	0.056	1.000	1.799	0.074	*
a5	-4.184	1.923	-0.068	0.999	-2.176	0.031	**
co_a1	15.868	1.057	0.463	1.000	15.009	0	****
(Intercept)	-462.906	29.764	?	?	-15.553	0	****

PerformanceVector

```

PerformanceVector:
root_mean_squared_error: 85.916 +/- 0.000
root_relative_squared_error: 0.481
correlation: 0.880
squared_correlation: 0.774

```

Regularization

- ⊕ Handle collinearity and overfitting by adding penalty term when fitting the model
- ⊕ Reasons for overfitting in regression
 - ▶ Too large coefficients, i.e. effects of attributes are over estimated
 - ▶ Too many attributes are included in the model
- ⊕ **Lasso** (least absolute shrinkage and selection operator) method
 - ▶ L1-norm regularization
 - ▶ $\text{Penalty} = \lambda (|b_1| + |b_2| + \dots + |b_m|)$; $\lambda > 0$
 - ▶ Shrink the coefficient of less important attributes to zero, so they can be eliminated from the model
 - ▶ Good for feature selection → to get a simpler model

⊕ Ridge method

- ▶ L2-norm regularization
- ▶ $\text{Penalty} = \lambda (b_1^2 + b_2^2 + \dots + b_m^2) ; \lambda > 0$
- ▶ Push all coefficients toward zero but not zero, so none is eliminated
- ▶ Allow correlated attributes to stay in the model but the effect from collinearity is reduced

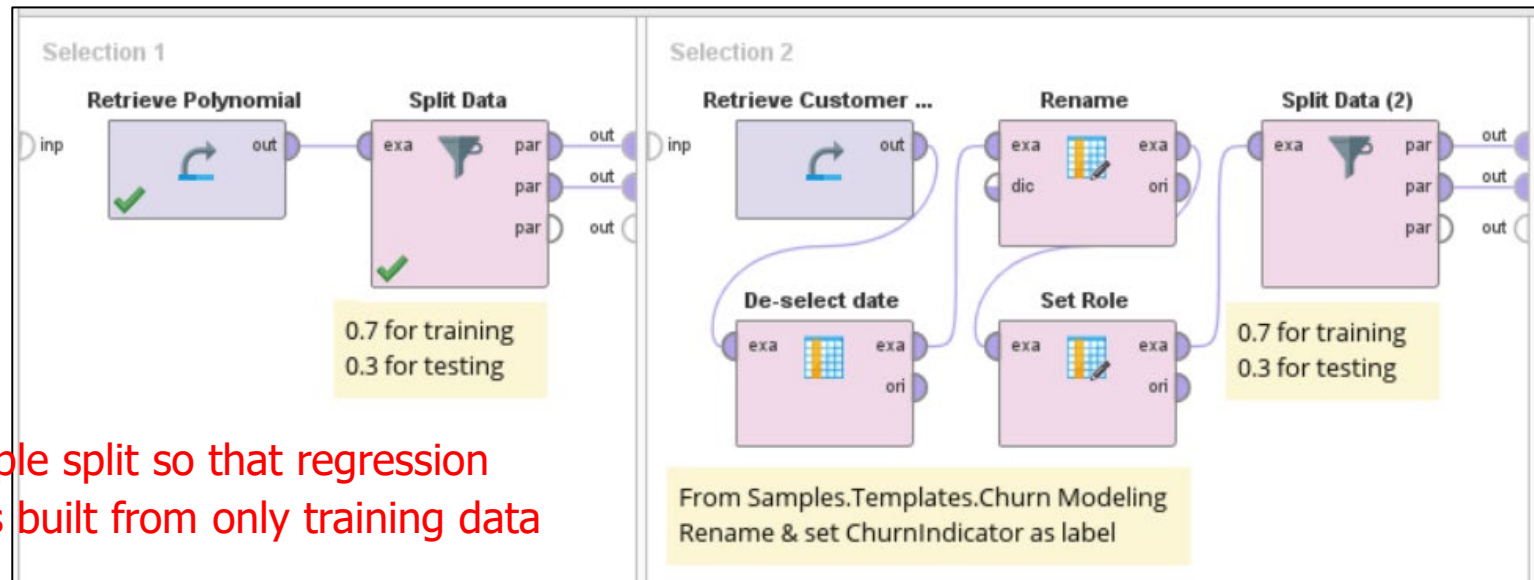
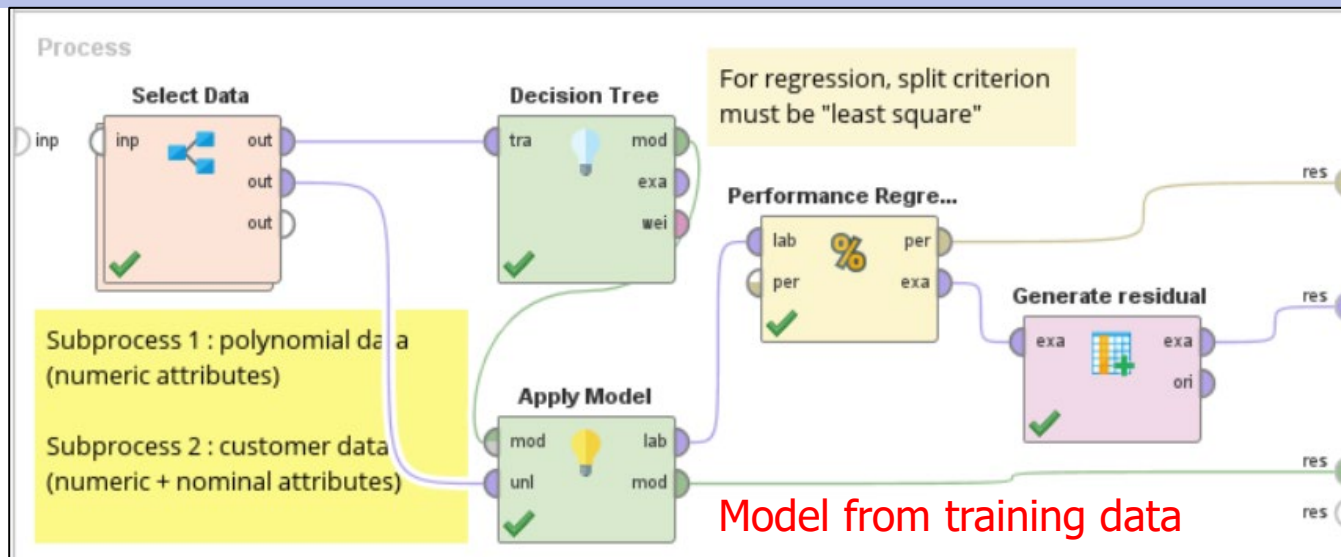
⊕ For both methods

- ▶ $\lambda = 0$ means no penalty
- ▶ Since there is no upper bound, setting a proper λ is difficult

Classification and Regression Tree (CART)

- ⊕ CART outputs a decision tree where leaf nodes are either classes (classification) or numeric values (regression), depending on the type of target variable Y
- ⊕ See Chapter 4 for tree construction
 - ▶ Only binary split
 - ▶ Best split for classification is based on Gini
 - ▶ Best split for regression is based on MSE or RMSE
 - Attribute that gives lowest squared error is chosen as tree node
 - Value at a leaf node = $\bar{y}$ from all records in that node
- ⊕ Can support both nominal & numeric attributes

Example (3) : decision tree for regression



RegressionTree

```

a2 > 7.040
|  a1 > 5.706: 533.035 {count=19}
|  a1 ≤ 5.706
|  |  a1 > 3.873: 289.533 {count=8}
|  |  a1 ≤ 3.873: 168.296 {count=14}
a2 ≤ 7.040
|  a2 > 2.457
|  |  a1 > 6.415: 251.813 {count=16}
|  |  a1 ≤ 6.415
|  |  |  a3 > 6.845: 166.430 {count=14}
|  |  |  a3 ≤ 6.845: 74.481 {count=31}
|  a2 ≤ 2.457
|  |  a2 > 0.851
|  |  |  a3 > 7.507: 76.075 {count=8}
|  |  |  a3 ≤ 7.507: 39.595 {count=12}
|  |  a2 ≤ 0.851: 10.468 {count=18}

```

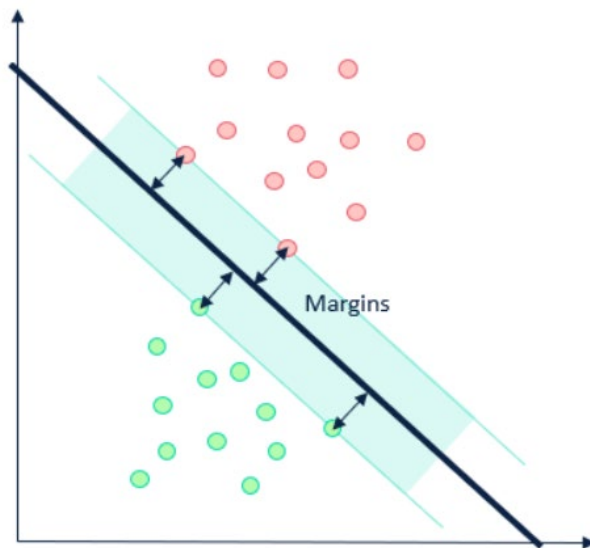
Row...	label	prediction(label)	a1	a2	a3
1	88.016	168.296	0.637	8.442	2.116
2	138.755	74.481	4.595	4.388	4.926
3	97.405	166.430	2.691	3.572	7.806
4	140.767	533.035	6.226	8.381	0.351
5	285.871	168.296	3.163	8.397	7.110
6	15.450	10.468	9.467	0.216	6.530
7	779.747	533.035	9.091	7.525	9.571
8	22.238	10.468	9.618	0.722	2.126
9	333.999	289.533	5.636	7.255	5.882
10	43.012	74.481	1.033	5.257	1.830

All testing records that fall into the same leaf node will have the same predicted value e.g. 533.035 ($= \bar{y}$ from 19 training records that fall into this node)

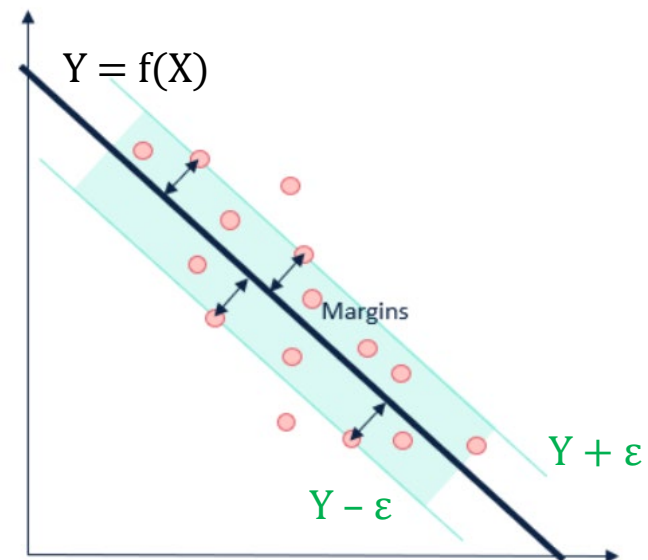
In practice, when using Validation operator, values in regression tree will be slightly different from predicted values because the final tree is built from full data set

Support Vector Regression (SVR)

- ⊕ Linear regression $\rightarrow$ fit $Y = f(X)$ that minimizes squared error
- ⊕ Support vector regression $\rightarrow$ fit a hyperplane that keeps points within an **acceptable margin ε**

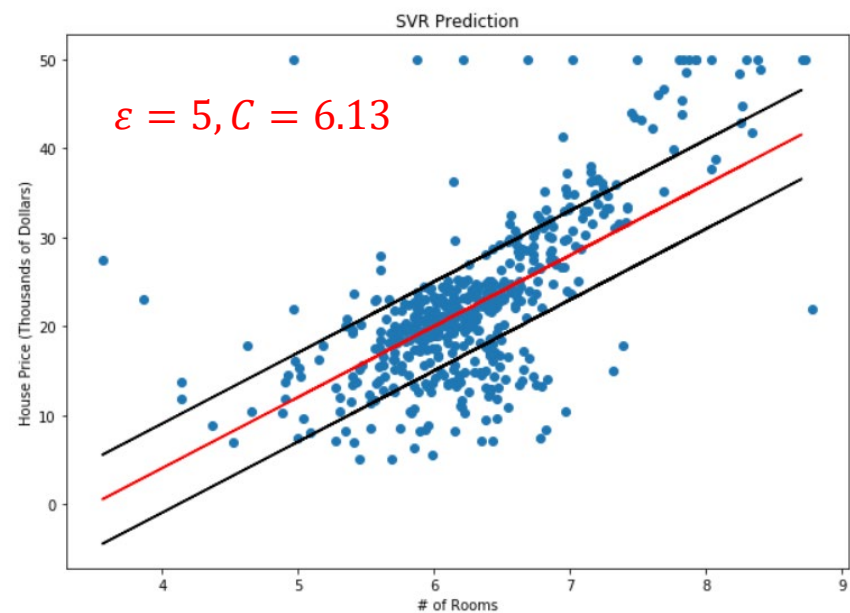
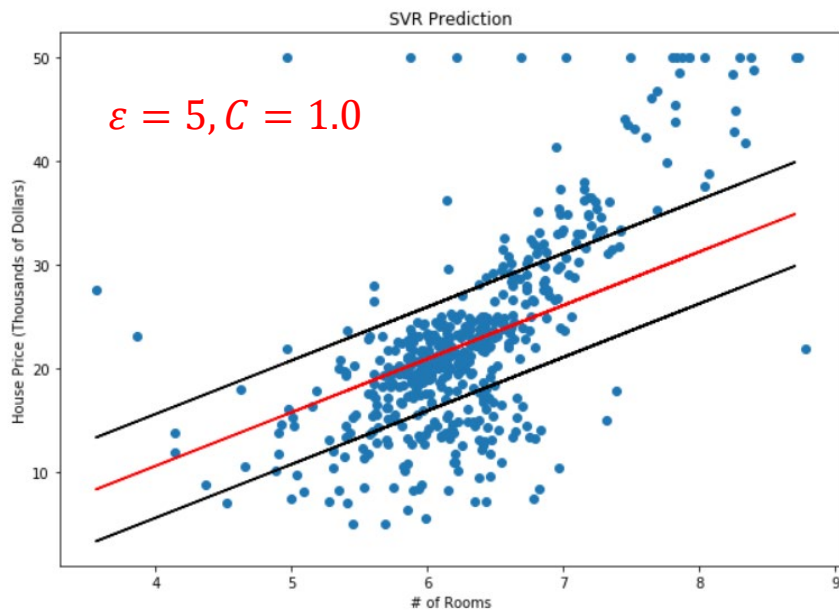


SVM : build hyperplane with widest margin to separate points from different classes



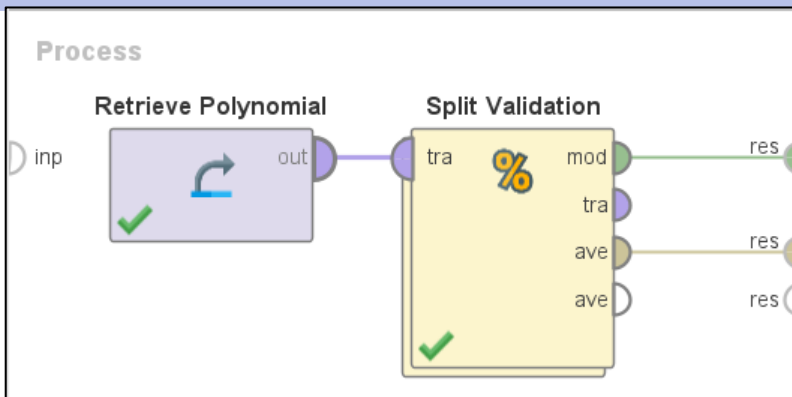
SVR : build hyperplane with given margin to keep most points within this margin

- ⊕ Not all points can be kept inside ε margin. Some are allowed to be outside the margin with some penalty $\rightarrow$ minimize total penalty
 - ▶ Like SVM, penalty term is controlled by parameter C
 - ▶ Big C = high penalty ; small C = low penalty
- ⊕ Interaction between ε and C affects the orientation of hyperplane

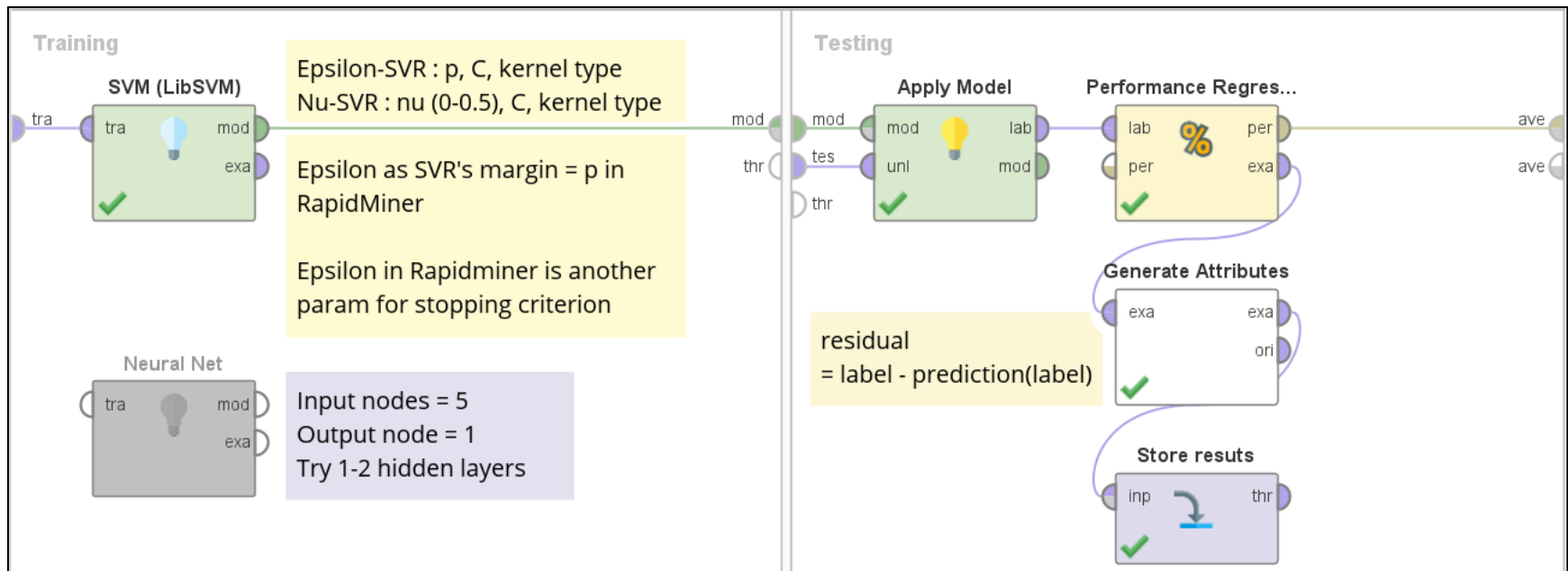


- ⊕ SVR can support nonlinear data via kernels and kernel trick
- ⊕ Famous implementation → **LibSVM** with 2 modules
 - ▶ **Epsilon-SVR (ϵ -SVR)**
 - ϵ = acceptable margin of deviation
 - C = penalty of too much deviation (being outside the margin)
 - ▶ **Nu-SVR (ν -SVR)**
 - ν = min support vectors = max points outside the margin
 - C = penalty of too much deviation (being outside the margin)
 - Nu-SVC uses ν instead of C (see SVM in Chapter 6), but Nu-SVR uses ν instead of ϵ
 - ▶ Epsilon-SVR focuses on controlling error while Nu-SVR focuses on controlling model complexity

Example (4) : SVR



Try using Optimization grid to find optimal parameters for SVR



Epsilon-SVR (C = 700, p = 80)

PerformanceVector

```
PerformanceVector:
root_mean_squared_error: 60.777 +/- 0.000
root_relative_squared_error: 0.367
correlation: 0.944
squared_correlation: 0.892
```

Kernel Model

```
Total number of Support Vectors: 46
Bias (offset): 260.903
```

```
w[a1] = 62714375.889
w[a2] = 69645875.080
w[a3] = 59859065.978
w[a4] = 45963848.205
w[a5] = 36755044.838
```

```
number of classes: 2
number of support vectors: 46
```

Nu-SVR (C = 100, nu = 0.38)

PerformanceVector

```
PerformanceVector:
root_mean_squared_error: 58.529 +/- 0.000
root_relative_squared_error: 0.543
correlation: 0.914
squared_correlation: 0.836
```

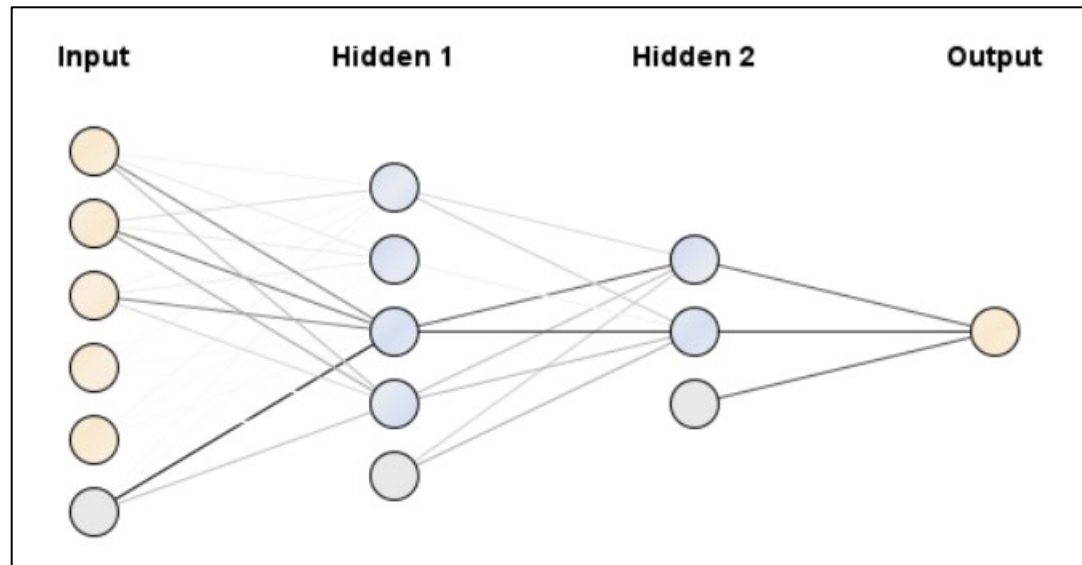
Kernel Model

```
Total number of Support Vectors: 80
Bias (offset): 265.876
```

```
w[a1] = 12515981.608
w[a2] = 14374595.735
w[a3] = 12035125.466
w[a4] = 9469652.596
w[a5] = 8205865.829
```

```
number of classes: 2
number of support vectors: 80
```

Neural network for regression



1 output node for numerical prediction

PerformanceVector

```
PerformanceVector:  
root_mean_squared_error: 40.780 +/- 0.000  
root_relative_squared_error: 0.228  
correlation: 0.974  
squared_correlation: 0.948
```



Read CSV

Set Role

Split Validation

SVR_nonlinear.csv

Locate Excel file and set $\text{role}(y) = \text{label}$



Linear Regression

SVM

Try polynomial degree 4
or RBF kernel

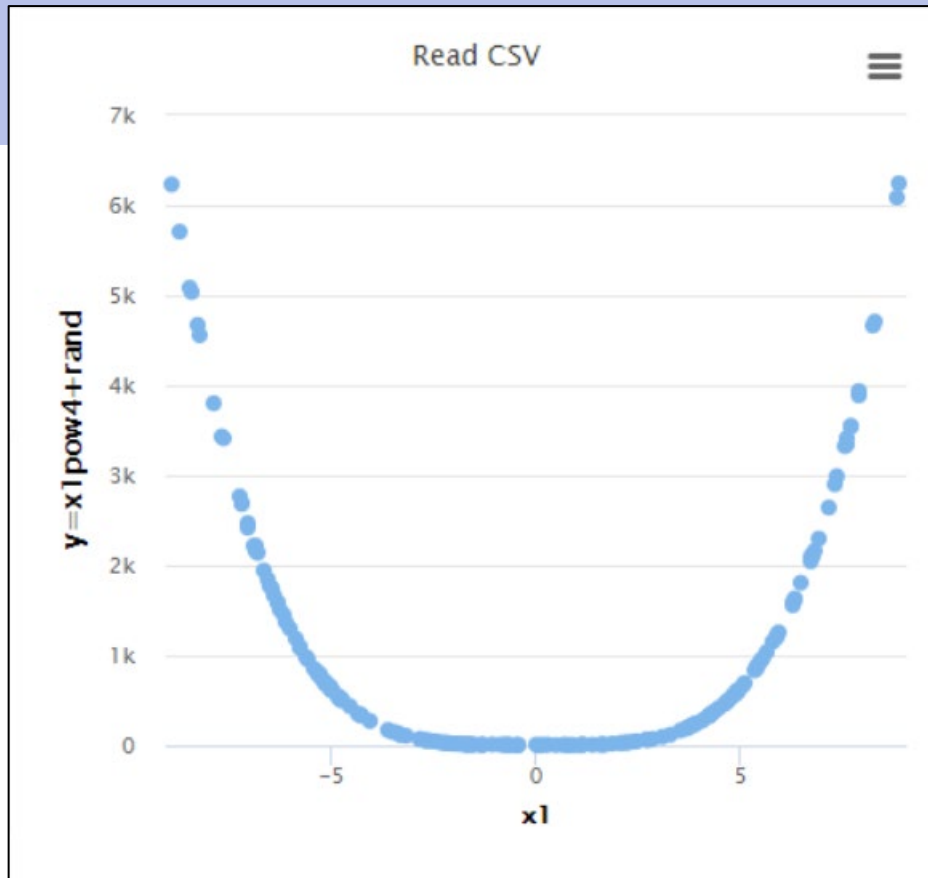
Testing

Apply Model

Performance Regres...

Generate residual

Store results



Linear regression performance

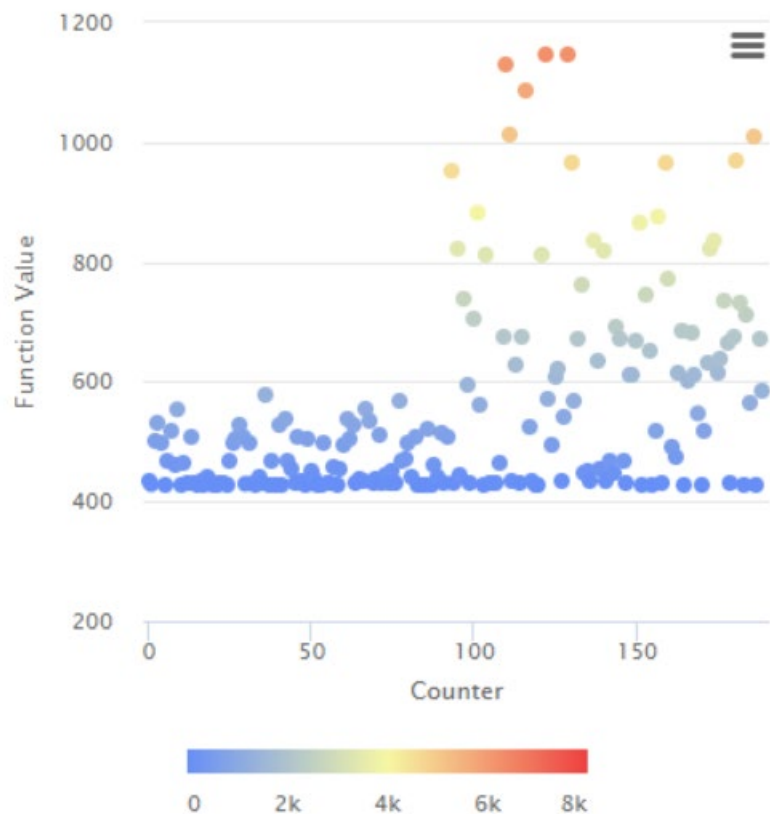
PerformanceVector

```
PerformanceVector:
root_mean_squared_error: 1444.598 +/- 0.000
root_relative_squared_error: 1.017
correlation: 0.000
squared_correlation: 0.000
```

Relationship between x1 and y
is not linear. So correlation = 0

Linear regression model

Attribute	Coefficient	Std. Error	Std. Coef...	Tolerance	t-Stat	p-Value	Code
x1	2.895	19.975	0.010	1	0.145	0.885	
(Intercept)	1031.401	98.700	?	?	10.450	0	****



Kernel Model

Total number of Support Vectors: 190

Bias (offset): -428.613

$w[x_1] = -10433011.082$

number of classes: 2

number of support vectors: 190

PerformanceVector

PerformanceVector:

root_mean_squared_error: 913.643 +/- 0.000

root_relative_squared_error: 0.669

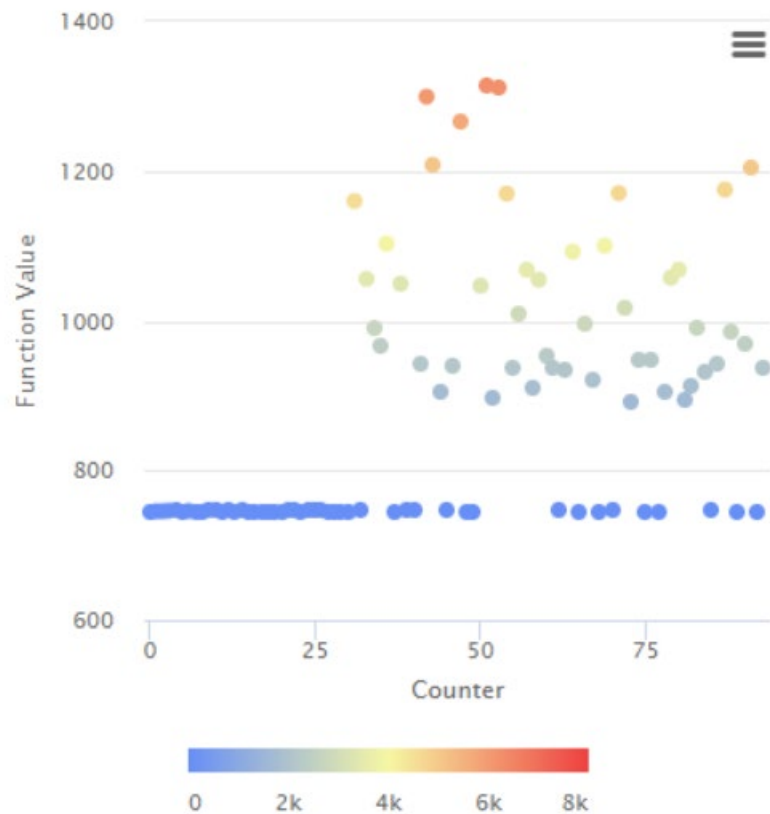
correlation: 1.000

squared_correlation: 1.000

Epsilon-SVR

Kernel = polynomial, degree 4

$C = 1000$, $p = 60$



Kernel Model

Total number of Support Vectors: 94

Bias (offset): -745.462

$w[x_1] = 6070166.201$

number of classes: 2

number of support vectors: 94

PerformanceVector

PerformanceVector:

root_mean_squared_error: 1054.646 +/- 0.000

root_relative_squared_error: 0.746

correlation: 1.000

squared_correlation: 1.000

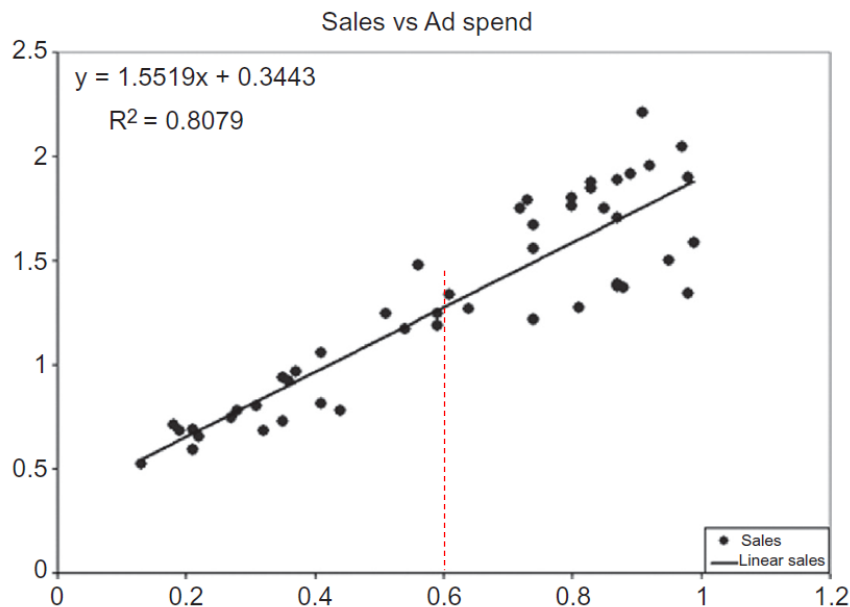
Nu-SVR

Kernel = polynomial, degree 4

$C = 1000$, $\nu = 0.46$

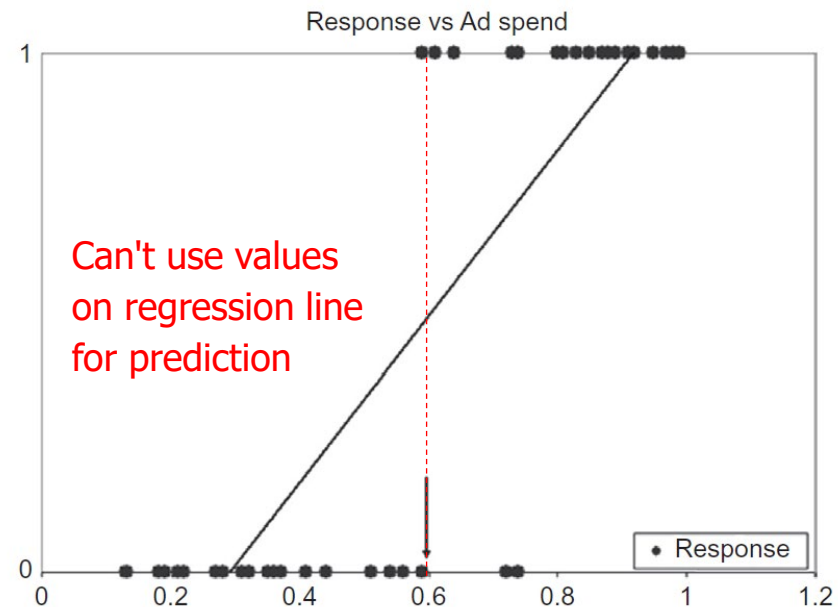
Logistic Regression

⚡ When Y = class label, linear regression no longer fits the data



Numeric prediction

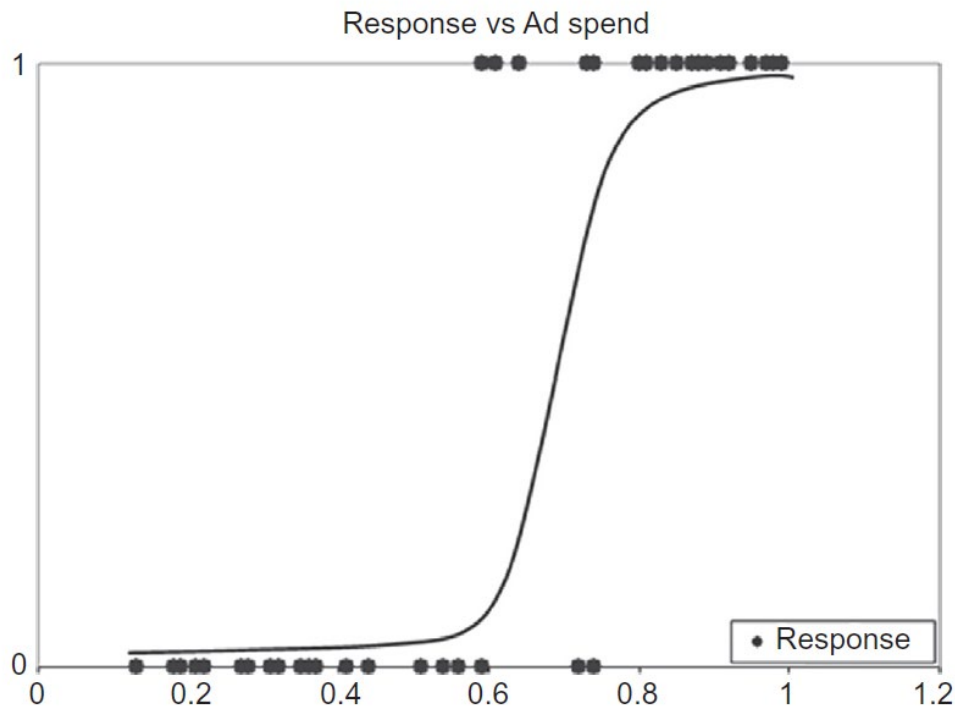
Ad spend = 0.6, predicted sales = value on regression line



Class prediction

Ad spend < 0.6, response = no
Ad spend = 0.6, response can be either yes or no. It is unpredictable

⊕ But S-curve (Sigmoid) can better fit most of the data



When Ad spend ≤ 0.6 ,
response is likely to be no

When Ad spend = 0.8,
response is likely to be yes

Logit and Logistic (Sigmoid) Functions

- ⊕ Consider binary classification where $Y = \text{yes (1) or no (0)}$

$$P(\text{yes}) = p ; P(\text{no}) = 1 - p ; \text{the odds of yes} = \frac{p}{1 - p}$$

- ⊕ **Logit** function of p = logarithm of the odds

$$\text{logit}(p) = \log\left(\frac{p}{1 - p}\right)$$

- ⊕ **Logistic or Sigmoid** function of p = inverse of logit function

$$\text{Sigmoid}(p) = \frac{e^p}{1 + e^p} ; e \approx 2.718$$

$$\text{Sigmoid}(\text{logit}(p)) = p$$

- ⊕ Logit function has linear relationship with X. So we can write

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = b_0 + b_1x$$

- ⊕ Given a regression model and a training record (x, y)
 - ▶ P(yes) or P(y = 1) can be calculated by
 - Substituting x in the model to get logit value
 - Calculate Sigmoid(logit) to get p
 - ▶ The **likelihood** of correct prediction for 1 record = $p^y (1-p)^{(1-y)}$
- ⊕ Model fitting = find optimal regression coefficients that maximizes the total likelihood for all training records
- ⊕ Can be extended to multiple attributes

Writing Logistic Regression Model

⊕ For linear regression

$$y = b_0 + b_1A_1 + b_2A_2 + \cdots + b_mA_m$$

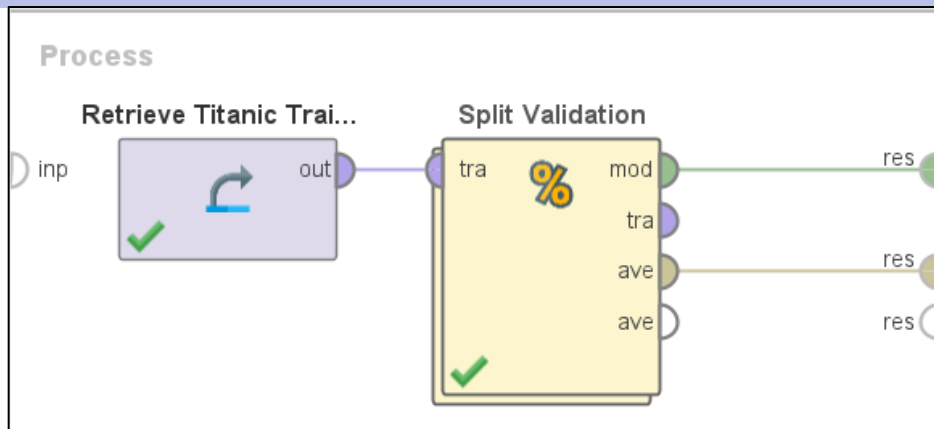
⊕ For logistic regression

$$\log\left(\frac{P(y = 1)}{1 - P(y = 1)}\right) = b_0 + b_1A_1 + b_2A_2 + \cdots + b_mA_m$$

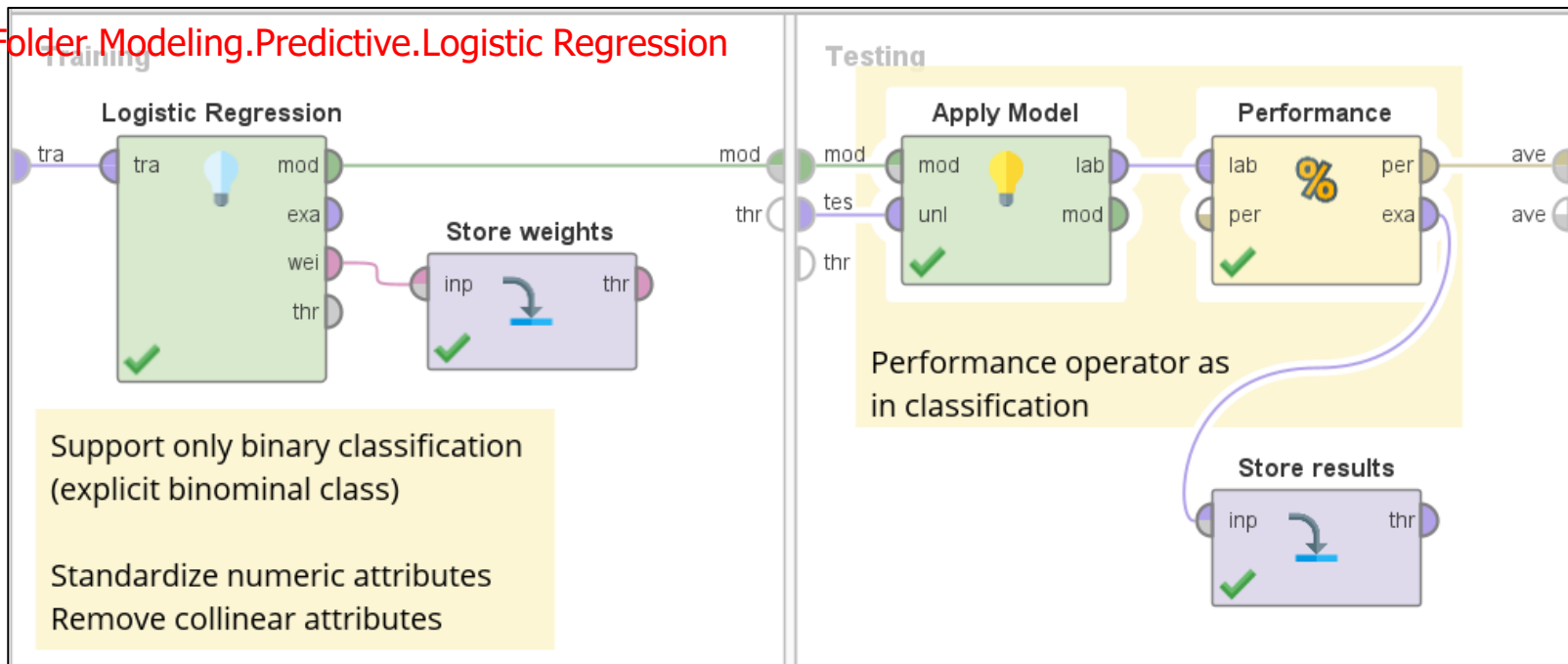
Or in terms of probability

$$P(y = 1) = \frac{e^{(b_0 + b_1A_1 + b_2A_2 + \cdots + b_mA_m)}}{1 + e^{(b_0 + b_1A_1 + b_2A_2 + \cdots + b_mA_m)}}$$

Example (6) : logistic regression



Folder Modeling.Predictive.Logistic Regression



Attribute	Coefficient	Std. Coefficient	Std. Error	z-Value	p-Value
Passenger Class.Second	1.240	1.240	0.291	4.259	0.000
Passenger Class.Third	1.857	1.857	0.277	6.704	0.000
Sex.Male	2.701	2.701	0.188	14.346	0
Age	0.030	0.394	0.007	4.021	0.000
No of Siblings or Spouses on Board	0.241	0.244	0.108	2.237	0.025
No of Parents or Children on Board	0.129	0.111	0.115	1.125	0.260
Passenger Fare	-0.002	-0.100	0.002	-0.840	0.401
Intercept	-3.349	-2.353	0.412	-8.123	0.000

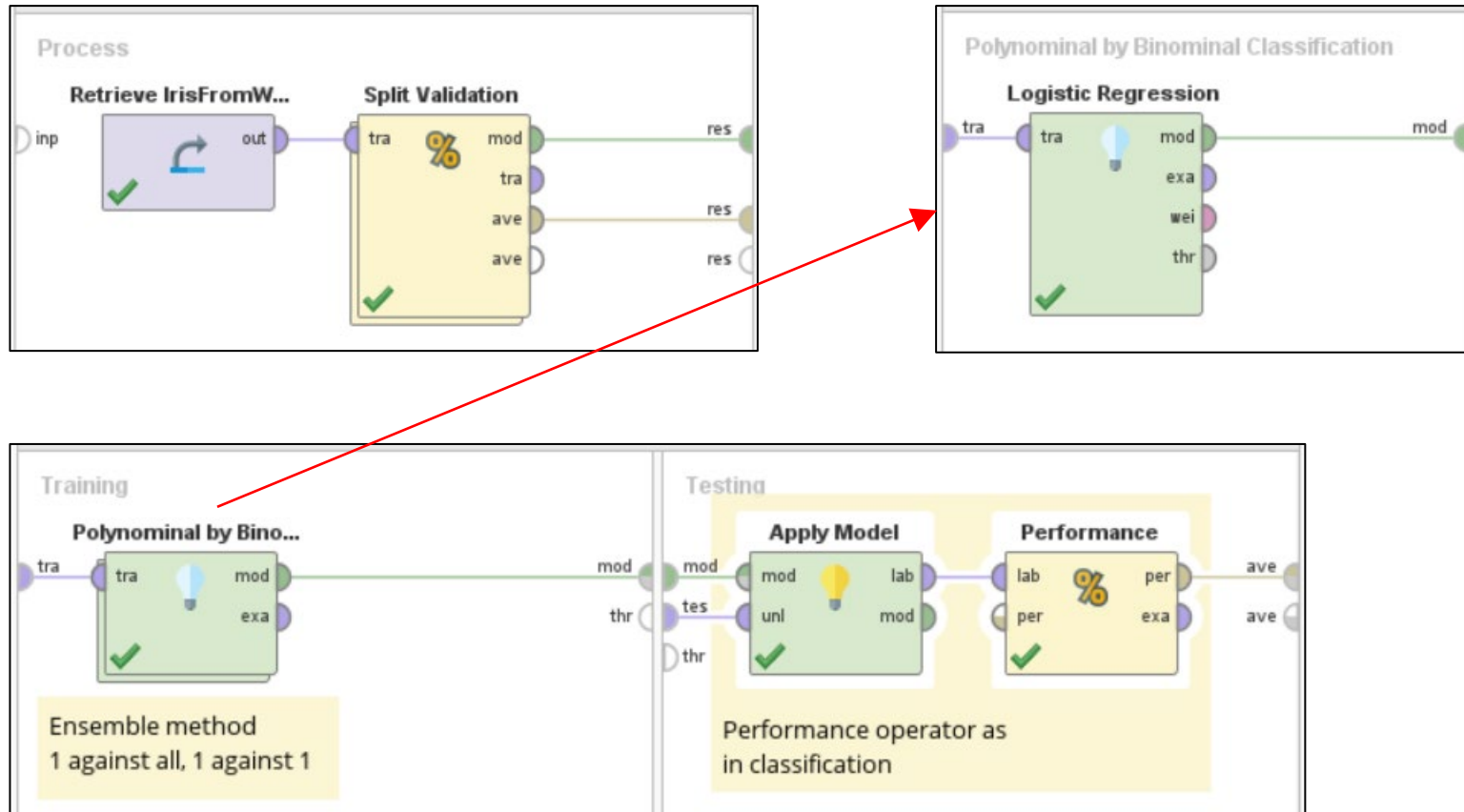
attribute	weight ↓
Sex = Male	2.701
Passenger Class = Third	1.857
Passenger Class = Second	1.240
Age	0.394
No of Siblings or Spouses on Board	0.244
No of Parents or Children on Board	0.111
Passenger Class = First	0
Sex = Female	0
Passenger Fare	-0.100

Nominal attributes are converted to numeric automatically

Z-Value and p-Value = statistical test on coefficient (similar to t-stat in linear regression)

P-value close to 0 = high significance

Example (7) : logistic regression with polynominal class



Summary

- ⊕ **Linear regression** is an old technique from statistics field
 - ▶ Support only numeric attributes and numeric target
 - ▶ Most calculation involves statistical test (t-stat, z-stat, p-value)
 - ▶ Model fitting is based on least squared error
- ⊕ Checking the usefulness of linear regression model
 - ▶ Normal distribution of residuals → requirement from stat theory
 - ▶ Coefficient of determination (r^2)
 - ▶ Potential collinearity problem
 - ▶ Scatter matrix is useful for preliminary check of relationship patterns between attributes

- ⊕ Linear regression assumes linear relationship between attributes and target
 - ▶ If the model doesn't fit the data, it can be because either they don't have any relationship or their relationship is not linear
 - ▶ For nonlinear relationship, we can use LibSVM instead

- ⊕ **CART** uses decision tree concept to build regression tree
 - ▶ Attributes with lowest squared error are chosen as tree nodes
 - ▶ Regression model is not a math function of attributes, but rather based on sets of conditions on attributes
 - ▶ Can support both numeric and nominal attributes
- ⊕ **LibSVM** uses SVM concept for regression
 - ▶ Model fitting is based on hyperplane & acceptable margin of errors
 - ▶ Use kernel trick to handle nonlinear relationship
 - ▶ But support only numeric attributes
- ⊕ No statistical assumption about residuals and collinearity for CART and LibSVM

- ⊕ **Logistic regression** extends linear regression for classification
 - ▶ Naturally support numeric attributes and binary class → **RapidMiner automatically converts nominal attributes to numeric**
 - ▶ Try to fit S-curve to the data by using linear relationship between attributes and logit (log of the odds for $y = \text{yes}$)
 - ▶ Model fitting is based on maximum likelihood of correct prediction
 - ▶ Model evaluation is the same as in classification task. If performance isn't good or model doesn't fit the data → try other classification methods